



An-Najah National University

Computer Engineering Department

Distributed Operating Systems

Bazar Bookstore Project

Lab 1 Report: Multi-tier Online Book Store

Prepared by:

Hoor Yaish

Deema Hazeem

2026

1. Introduction

This report describes the implementation of Part 1 of the Bazar Bookstore project for the Distributed Operating Systems course. The goal of Part 1 was to build a small online bookstore using a multi-tier microservices architecture. The system consists of a frontend service, a catalog service, and an order service. All services communicate using REST API calls, and persistent data is stored using SQLite databases.

The bookstore supports three main user operations: searching books by topic, requesting information about a book by item number, and purchasing a book. The implementation was containerized using Docker and managed using Docker Compose.

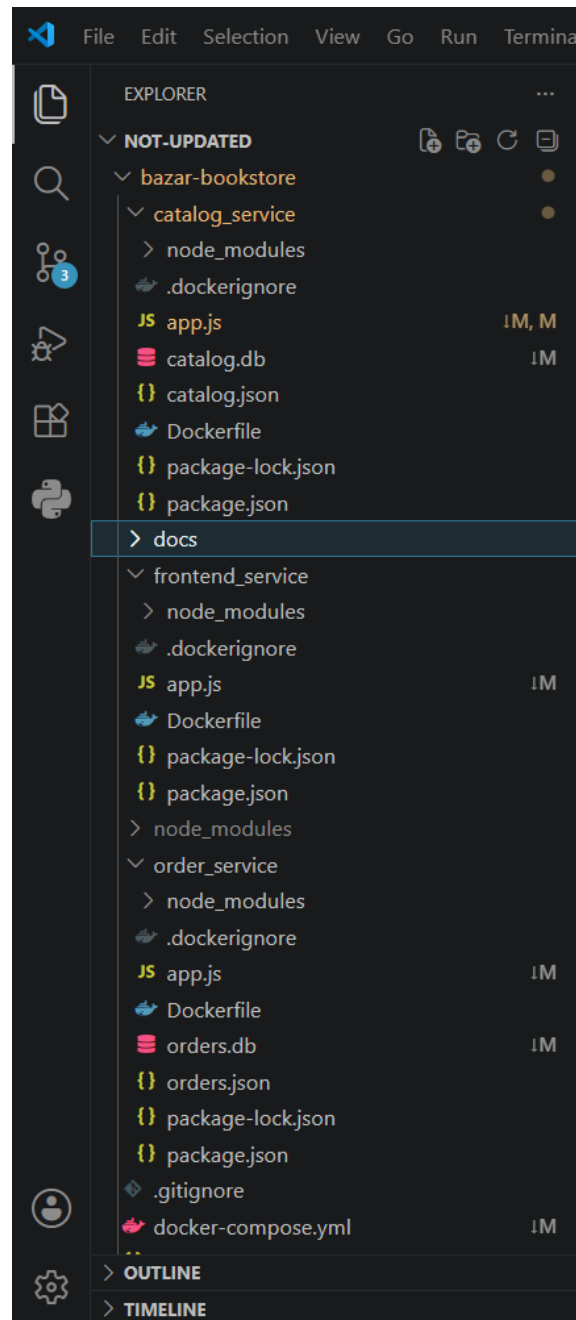


Figure 1: Project hierarchy showing the frontend, catalog, and order services.

2. System Architecture

The Part 1 system contains three main services:

Service	Responsibility	Port
Frontend service	Receives client requests and forwards them to the correct backend service.	5000
Catalog service	Stores book information and supports search, info, and update operations.	5001
Order service	Handles purchase requests and stores order records.	5002

The request flow is simple and follows the multi-tier model:

Client -> Frontend Service -> Catalog Service / Order Service

```
GET /search/:topic -> Frontend -> Catalog
GET /info/:id      -> Frontend -> Catalog
POST /purchase/:id -> Frontend -> Order -> Catalog
```

3. Technologies Used

Technology	Use in the Project
Node.js	Runtime environment for the services.
Express.js	Creating REST API endpoints.
Axios	Sending HTTP requests between services.
SQLite	Persistent storage for catalog and order data.
Docker	Containerizing each service.
Docker Compose	Building and running the three containers together.
Postman / Browser	Testing the REST API endpoints.

4. Service Implementation

4.1 Frontend Service

The frontend service is the entry point of the system. It exposes endpoints to the client and forwards requests to either the catalog service or the order service. It does not access the database directly. This separation keeps the frontend lightweight and follows the microservices design.

Endpoint	Method	Purpose
/search/:topic	GET	Forward a search request to the catalog service.
/info/:id	GET	Forward a book information request to the catalog service.
/purchase/:id	POST	Forward a purchase request to the order service.

4.2 Catalog Service

The catalog service maintains the bookstore catalog. It stores each book with its item number, title, topic, price, and available quantity. The service creates the books table if it does not already exist and inserts the initial four books required for Part 1.

Endpoint	Method	Purpose
/search/:topic	GET	Return all books that match a topic.
/info/:id	GET	Return title, quantity, and price for a book.
/update/:id	PUT	Update the quantity or price of a book.

4.3 Order Service

The order service handles purchase requests. When a purchase request is received, it first asks the catalog service for the book information. If the item is in stock, the order service calls the catalog update endpoint to decrease the quantity by one and then stores the order in the orders database.

Purchase flow:

1. Receive POST /purchase/:id
2. Query catalog service using GET /info/:id
3. Check if quantity > 0
4. Decrement quantity using PUT /update/:id
5. Insert the order into SQLite
6. Return a success message

5. Data Persistence

Persistent storage was implemented using SQLite. The catalog service stores book information in catalog.db, and the order service stores purchase records in orders.db. This ensures that catalog and order data are maintained on disk instead of being stored only in memory.

Database	Used By	Stored Data
catalog.db	Catalog service	Book id, title, topic, price, and quantity.
orders.db	Order service	Order id, item id, title, and purchase time.

6. Docker Deployment

Each service was containerized using a Dockerfile. Docker Compose was used to build and run the three containers together. All containers were connected to the same Docker bridge network so that the services could communicate using their service names.

```
docker compose down --remove-orphans
docker compose up --build
```

```
PROBLEMS OUTPUT DEBUG CONSOLE TERMINAL PORTS

#26 [order] resolving provenance for metadata file
#26 DONE 0.0s

#27 [frontend] resolving provenance for metadata file
#27 DONE 0.0s

#28 [catalog] resolving provenance for metadata file
#28 DONE 0.0s
[+] up 7/7
✓ Image bazar-bookstore-order Built 2.7s
✓ Image bazar-bookstore-frontend Built 2.7s
✓ Image bazar-bookstore-catalog Built 2.7s
✓ Network bazar-bookstore_bazar-network Created 0.1s
✓ Container bazar-catalog-service Created 0.1s
✓ Container bazar-order-service Created 0.1s
✓ Container bazar-frontend-service Created 0.1s
Attaching to bazar-catalog-service, bazar-frontend-service, bazar-order-service
bazar-catalog-service | Catalog service running on port 5001
bazar-catalog-service | Connected to catalog database.
bazar-order-service | Order service running on port 5002
bazar-order-service | Connected to orders database.
bazar-frontend-service | Frontend running on port 5000

View in Docker Desktop View Config Enable Watch Detach
```

Figure 2: Terminal output showing that the three Lab 1 services started successfully.

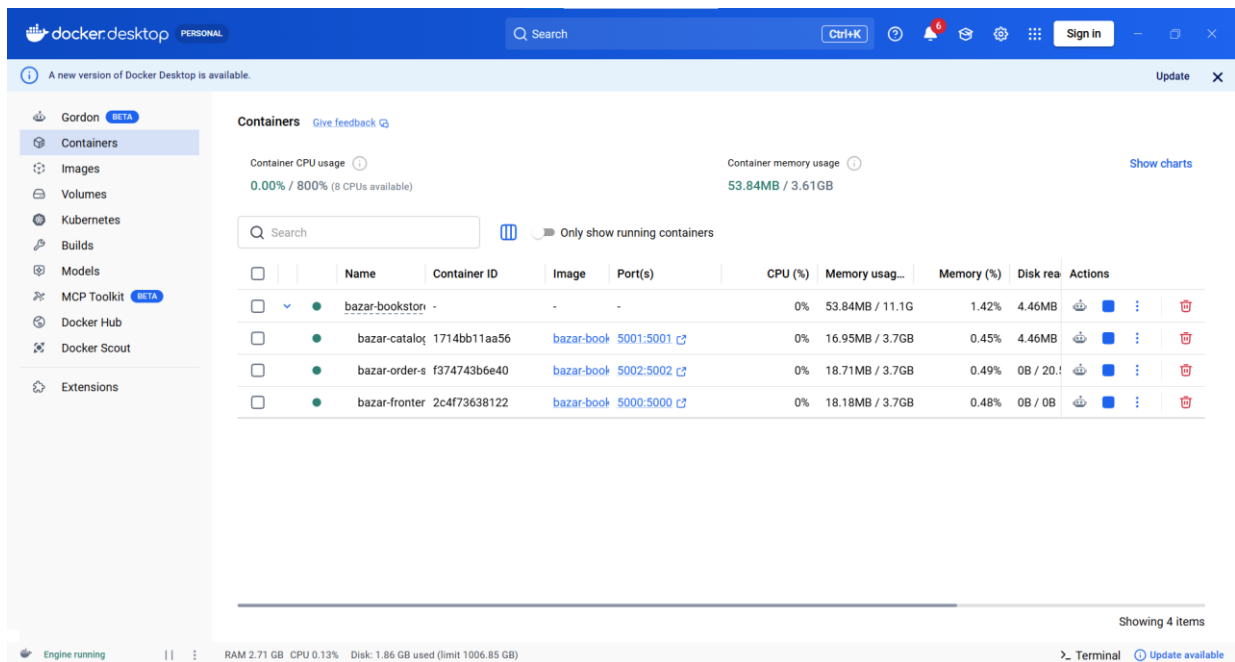


Figure 3: Docker Desktop showing the three Part 1 containers running.

7. Testing and Output

The system was tested using Postman and Docker logs. The tests demonstrate that search, info, and purchase operations work correctly, and that a successful purchase decreases the book quantity.

7.1 Search Request

The search request returns all books that belong to the distributed systems topic.

```
GET http://localhost:5000/search/distributed%20systems
```

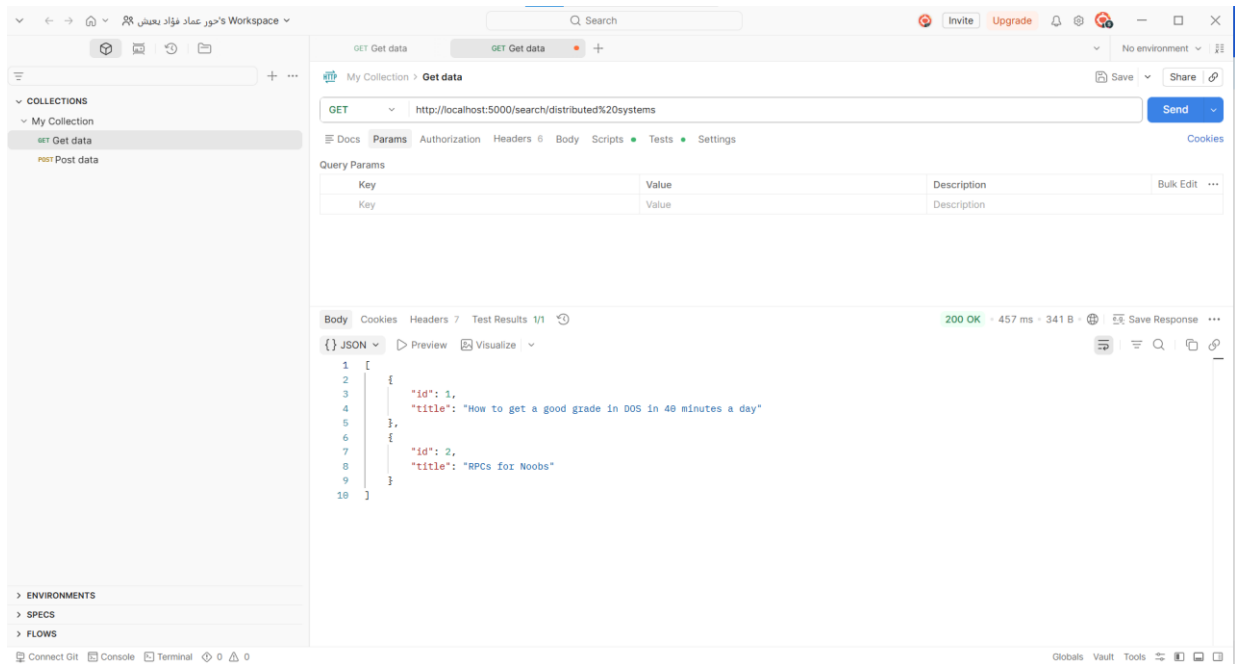


Figure 4: Search request returning books from the distributed systems topic.

7.2 Info Request Before Purchase

The info request returns the details of a book using its item number. In this test, item 2 had quantity 5 before the purchase.

GET `http://localhost:5000/info/2`

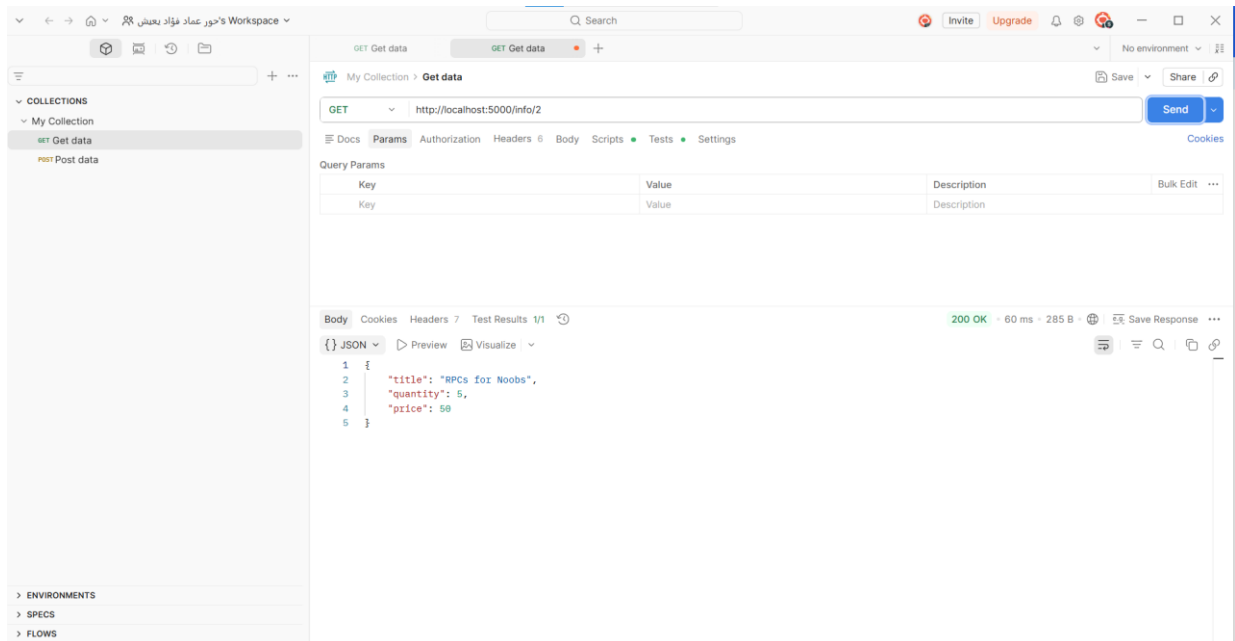


Figure 5: Info request for item 2 before purchase.

7.3 Purchase Request

The purchase request buys item 2. The order service checks the catalog, verifies that the book is in stock, decreases the quantity, stores the order, and returns a success message.

POST `http://localhost:5000/purchase/2`

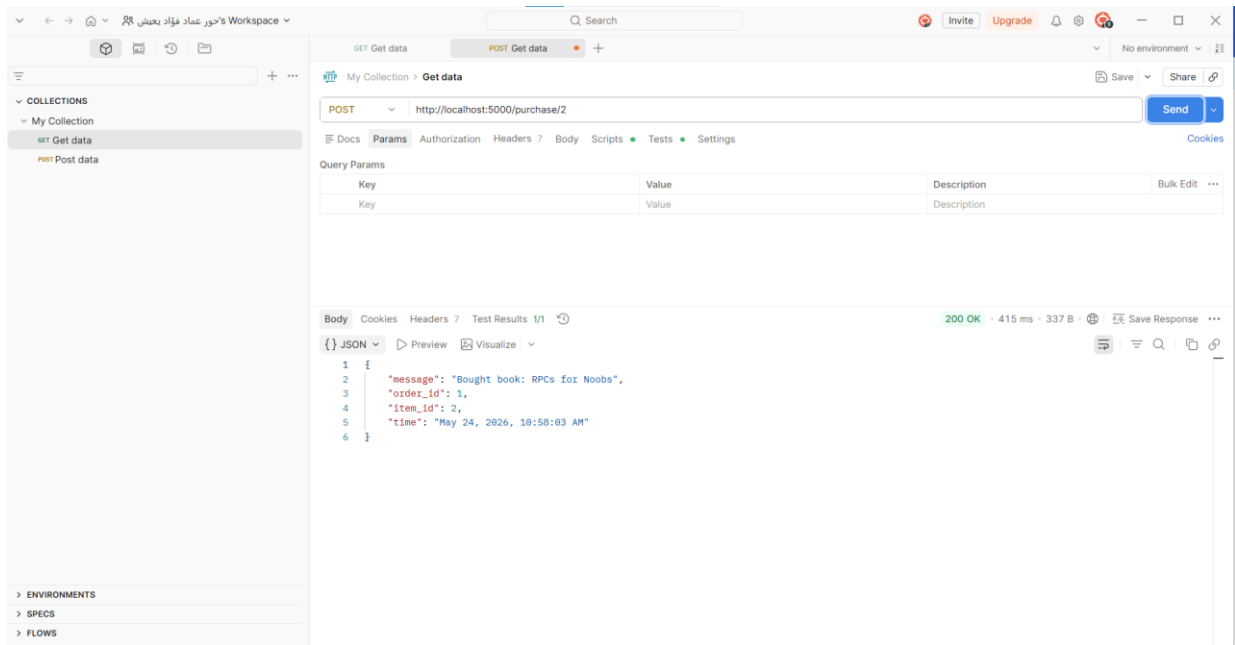


Figure 6: Successful purchase request for item 2.

7.4 Quantity Decrease After Purchase

After the purchase, another info request shows that the quantity of item 2 decreased from 5 to 4.

GET <http://localhost:5000/info/2>

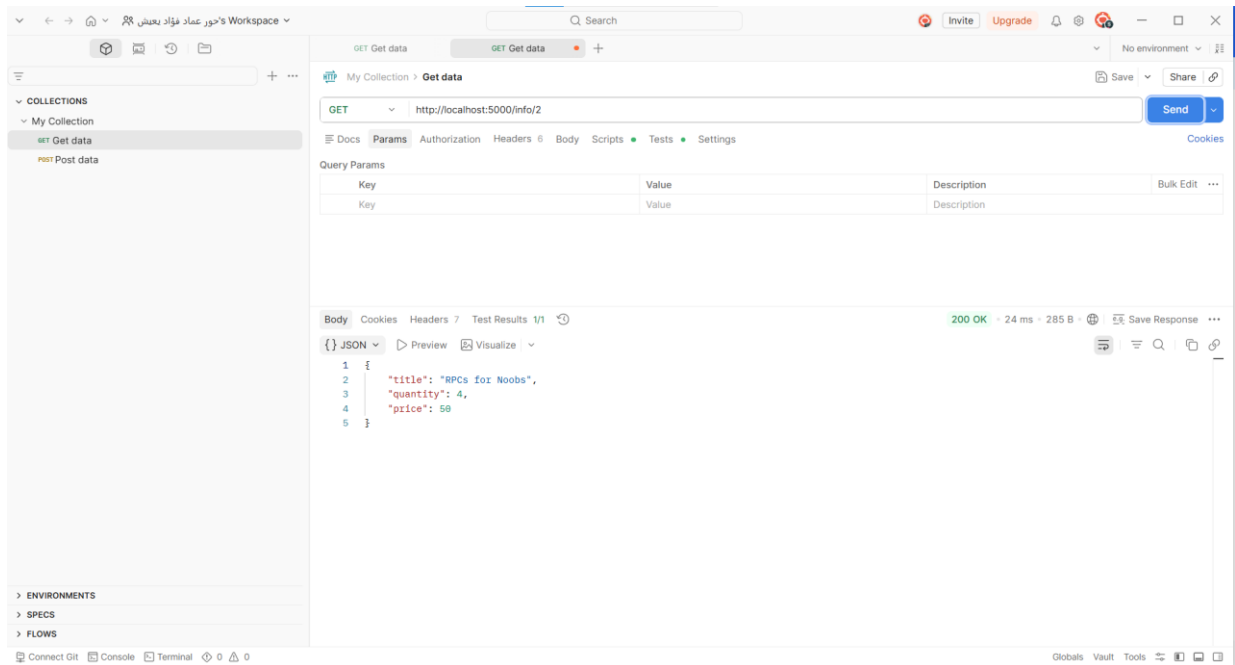


Figure 7: Info request after purchase showing that the quantity decreased.

7.5 Runtime Logs

The frontend logs show the complete flow: search request, info request, purchase request, and the updated quantity after purchase.

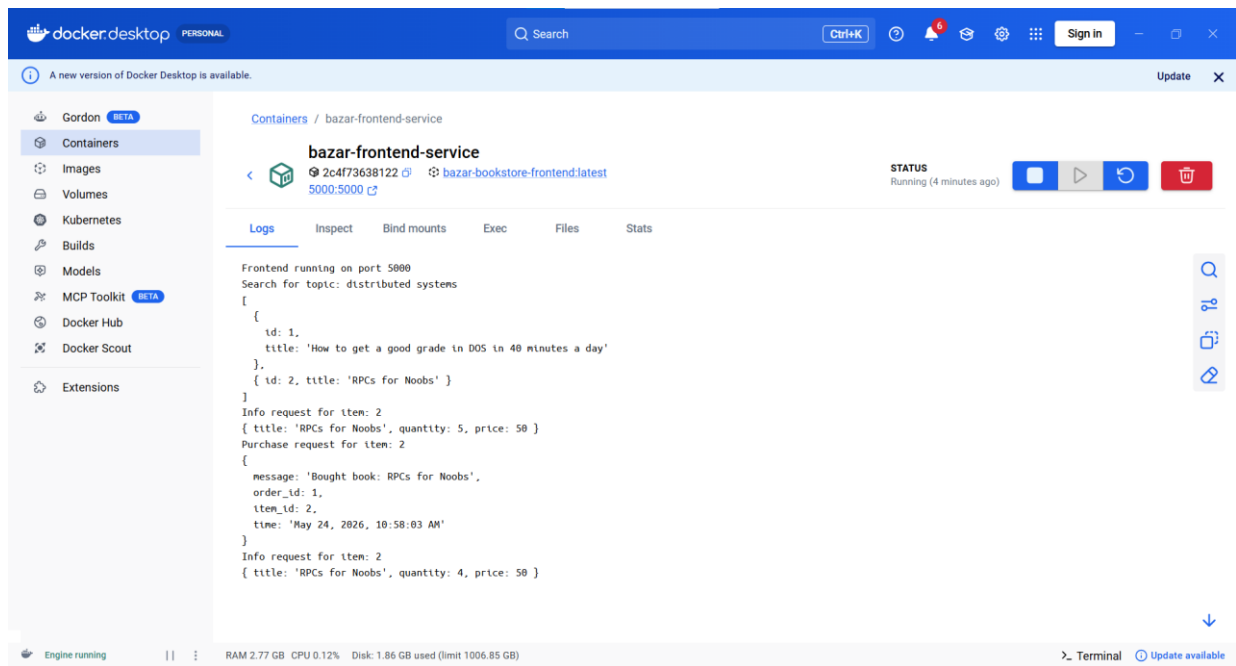


Figure 8: Frontend logs showing the search, info, purchase, and updated quantity outputs.

8. Design Tradeoffs

The design uses three separate microservices instead of one monolithic application. This makes the system more modular and easier to extend, but it requires service-to-service communication using HTTP calls.

SQLite was selected because it is lightweight and suitable for this lab. A heavier database would add unnecessary complexity for a small project.

Docker Compose was used to simplify deployment. Instead of starting each service manually, the whole system can be started with one command.

9. Known Limitations

The Lab 1 system has the following known limitations:

- There is only one catalog service and one order service, so there is no replication in Part 1.
- There is no caching, so every info request must contact the catalog service.
- The frontend is a single entry point and is not replicated.
- The system uses simple REST communication and does not include advanced fault tolerance.
- Concurrent purchases of the same item may require stronger transaction handling in a real production system.

10. Possible Improvements

Possible improvements and extensions include:

- Add caching in the frontend to reduce read latency.
- Replicate the catalog and order services to improve scalability and availability.
- Add load balancing between replicas.
- Add synchronization between replicated databases.
- Add better error handling, health checks, and automated tests.

11. How to Run the Project

Open a terminal in the project folder and run:

```
docker compose down --remove-orphans
docker compose up --build
```

Then test the system using these endpoints:

Operation	Method and URL
Search	GET http://localhost:5000/search/distributed%20systems
Info	GET http://localhost:5000/info/2
Purchase	POST http://localhost:5000/purchase/2

12. Conclusion

Part 1 successfully implements the required multi-tier online bookstore system. The project contains a frontend service, a catalog service, and an order service. The system supports search, info, and purchase operations through REST APIs. SQLite is used for persistent data storage, and Docker Compose is used to run the services as separate containers. The testing outputs prove that the services run correctly, book information can be retrieved, purchases are processed successfully, and stock quantity decreases after a purchase.